



Data Structures

Jasraj Meena

Assistant Professor

Department of Information Technology

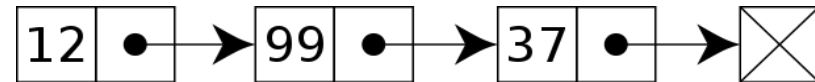
Delhi Technological University



Linked List [1]

- **Linked Lists [1]:**

- In computer science, a **linked list** is a linear collection of data elements whose order is not given by their physical placement in memory.
- Instead, each element points to the next. It is a data structure consisting of a collection of nodes which together represent a sequence [2] .





Linked List [1]

- ❖ **Drawback of Sequential Storage** (to represent Stack and Queue):
 - ❖ A fixed amount of storage remains allocated to the stack or queue even when structure is actually using a smaller amount or possibly no storage at all.
 - ❖ Possibility of overflow.
- ❖ **Advt. of Linked List:**
 - ❖ Dynamic Data Structure
 - ❖ Efficient Memory Utilization
 - ❖ Insertion and Deletion are easier and efficient
- ❖ **Dis. Advt.**
 - ❖ More Memory required

Linked List [1]

◆ Linked List [1]:

- ❖ Each item in the list is called a **node** and contains **two fields**, an **information field** and a **next address field**.
- ❖ The next address field of the last node in the list contains a special value known as null, which is not a valid address.
- ❖ The **null pointer** is used to signal the end of a list.



- ❖ The list with **no nodes** on it is called the **empty list** or the null list.



Linked List

◆ Linked List using C++:

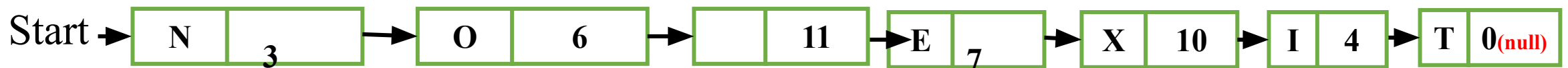
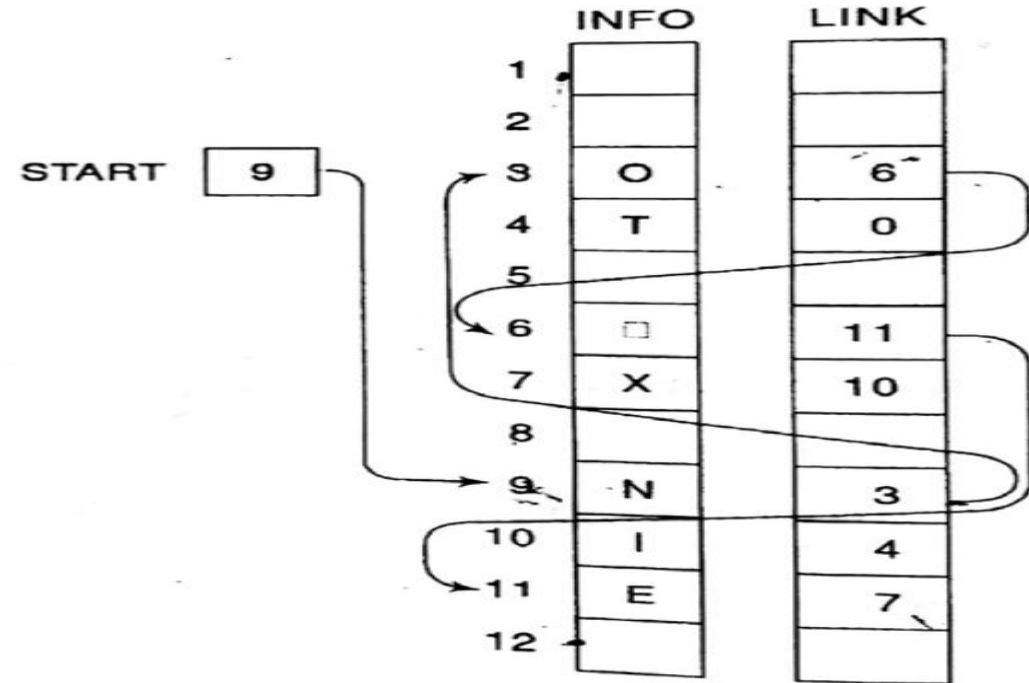
```
struct Node
{
    int info;
    struct Node *ptr;
};
```

```
int main()
{
    struct Node* start = NULL;
    return 0;
}
```

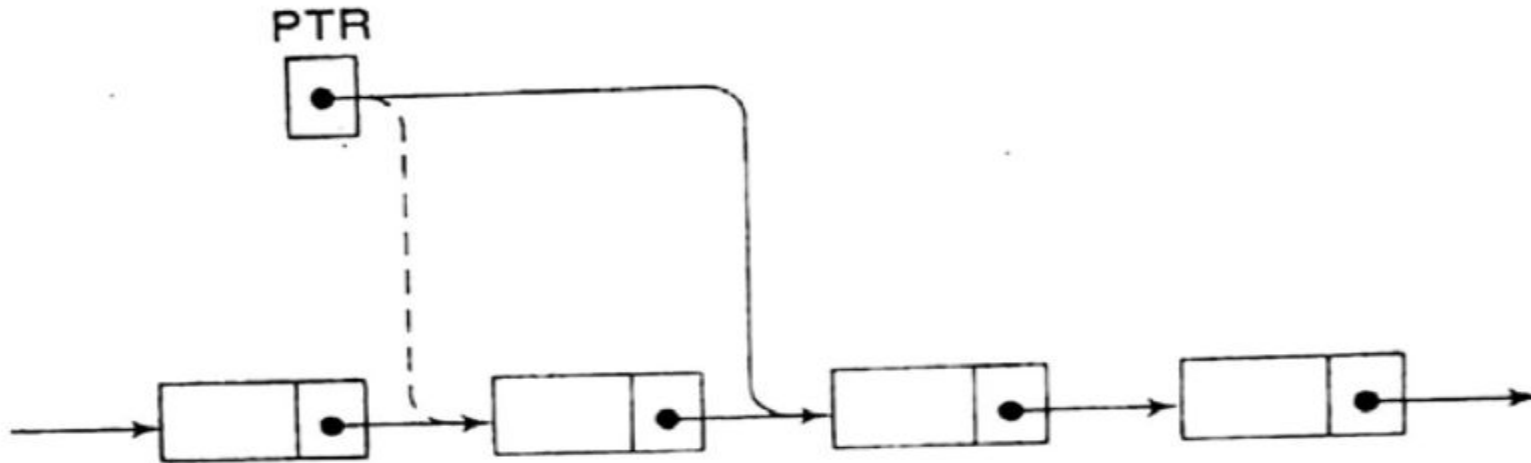
Linked List Representation in Memory [1]



START = 9, so INFO[9] = N is the first character.
LINK[9] = 3, so INFO[3] = O is the second character.
LINK[3] = 6, so INFO[6] = □ (blank) is the third character.
LINK[6] = 11, so INFO[11] = E is the fourth character.
LINK[11] = 7, so INFO[7] = X is the fifth character.
LINK[7] = 10, so INFO[10] = I is the sixth character.
LINK[10] = 4, so INFO[4] = T is the seventh character.
LINK[4] = 0, the NULL value, so the list has ended.



Traversing a Linked List [1]



$\text{PTR} := \text{LINK}[\text{PTR}]$



Linked List

◆ Linked List using C++:

```
struct Node
{
    int info;
    struct Node *ptr;
};

int main()
{
    struct Node* start = NULL;
    cout<<"The linked list : "<<endl;
    displayLinkedList(start);
    return 0;
}
```

```
void displayLinkedList(struct Node *node)
{
    //traverse the list to display each node
    while (node != NULL)
    {
        cout<<node->info<<".....>";
        node = node->ptr;
    }

    if(node== NULL)
        cout<<"null";
}
```




Searching a Linked List [1]

❖ List is Unsorted:

SEARCH(INFO, LINK, START, ITEM, LOC)

LIST is a linked list in memory. This algorithm finds the location LOC of the node where ITEM first appears in LIST, or sets LOC = NULL.

1. Set PTR := START.
2. Repeat Step 3 while PTR $\neq$ NULL:
3. If ITEM = INFO[PTR], then:
 Set LOC := PTR, and Exit.
 Else:
 Set PTR := LINK[PTR]. [PTR now points to the next node.]
 [End of If structure.]
 [End of Step 2 loop.]
4. [Search is unsuccessful.] Set LOC := NULL.
5. Exit.

Searching a Linked List [1]

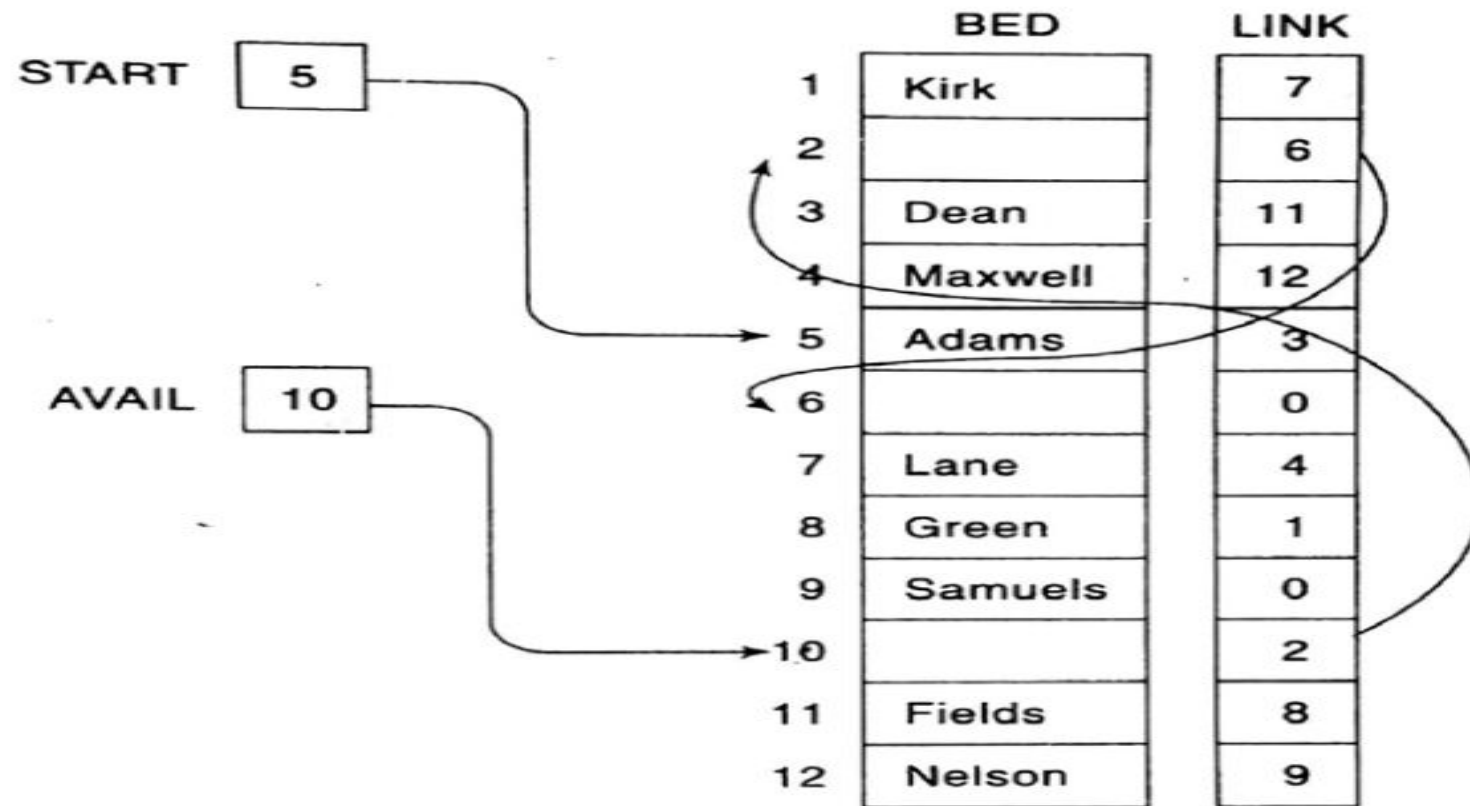
❖ List is Sorted:

SRCHSL(INFO, LINK, START, ITEM, LOC)

LIST is a sorted list in memory. This algorithm finds the location LOC of the node where ITEM first appears in LIST, or sets $LOC \leftarrow NULL$.

1. Set $PTR \leftarrow START$.
2. Repeat Step 3 while $PTR \neq NULL$:
3. If $ITEM < INFO[PTR]$, then:
 Set $PTR \leftarrow LINK[PTR]$. [PTR now points to next node.]
 Else if $ITEM = INFO[PTR]$, then:
 Set $LOC \leftarrow PTR$, and Exit. [Search is successful.]
 Else:
 Set $LOC \leftarrow NULL$, and Exit. [ITEM now exceeds $INFO[PTR]$.]
 [End of If structure.]
 [End of Step 2 loop.]
4. Set $LOC \leftarrow NULL$.
5. Exit.

Memory Collection and Garbage Collection [1]



Memory Collection and Garbage Collection [1]



❖ Garbage Collection:

- ❖ The operating system of a computer may periodically collect all the deleted space onto the free storage list. Any technique which does this collection is called garbage collection.
- ❖ Garbage collections usually takes place in two steps:
 - ❖ Step1: Tagging those cells which are currently in use in all lists.
 - ❖ Step2: Collecting all untagged space onto the free storage list (through memory).

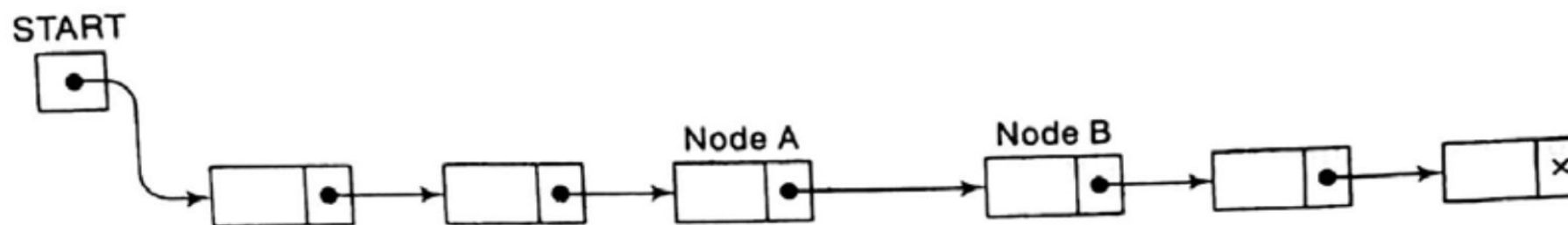


Linked List [1]

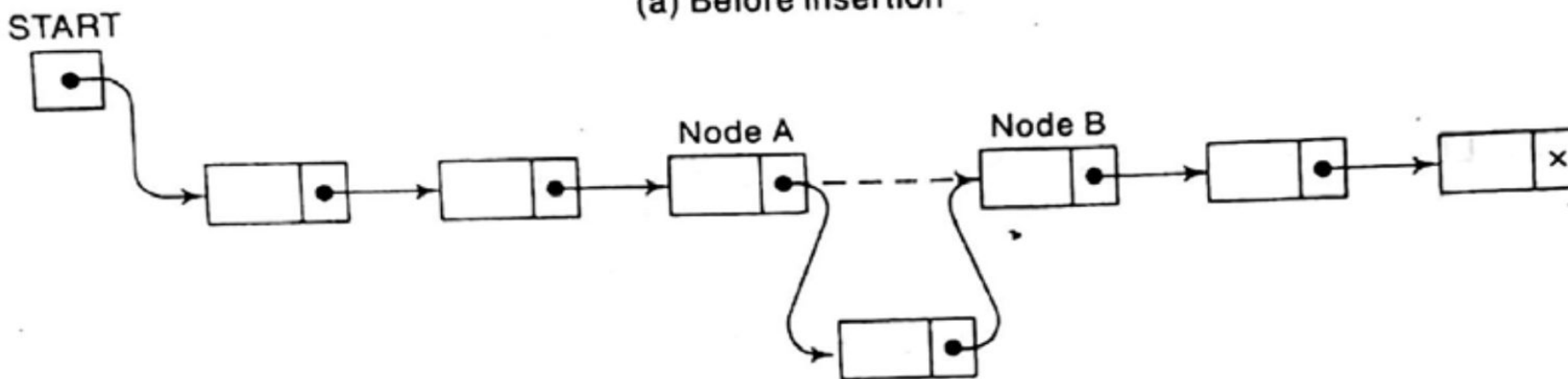
❖ Overflow and Underflow:

- ❖ Overflow occurs with our linked list when **AVAIL = NULL**
- ❖ Underflow will occurs with our linked list when **START = NULL**

Insertion in Linked List [1]

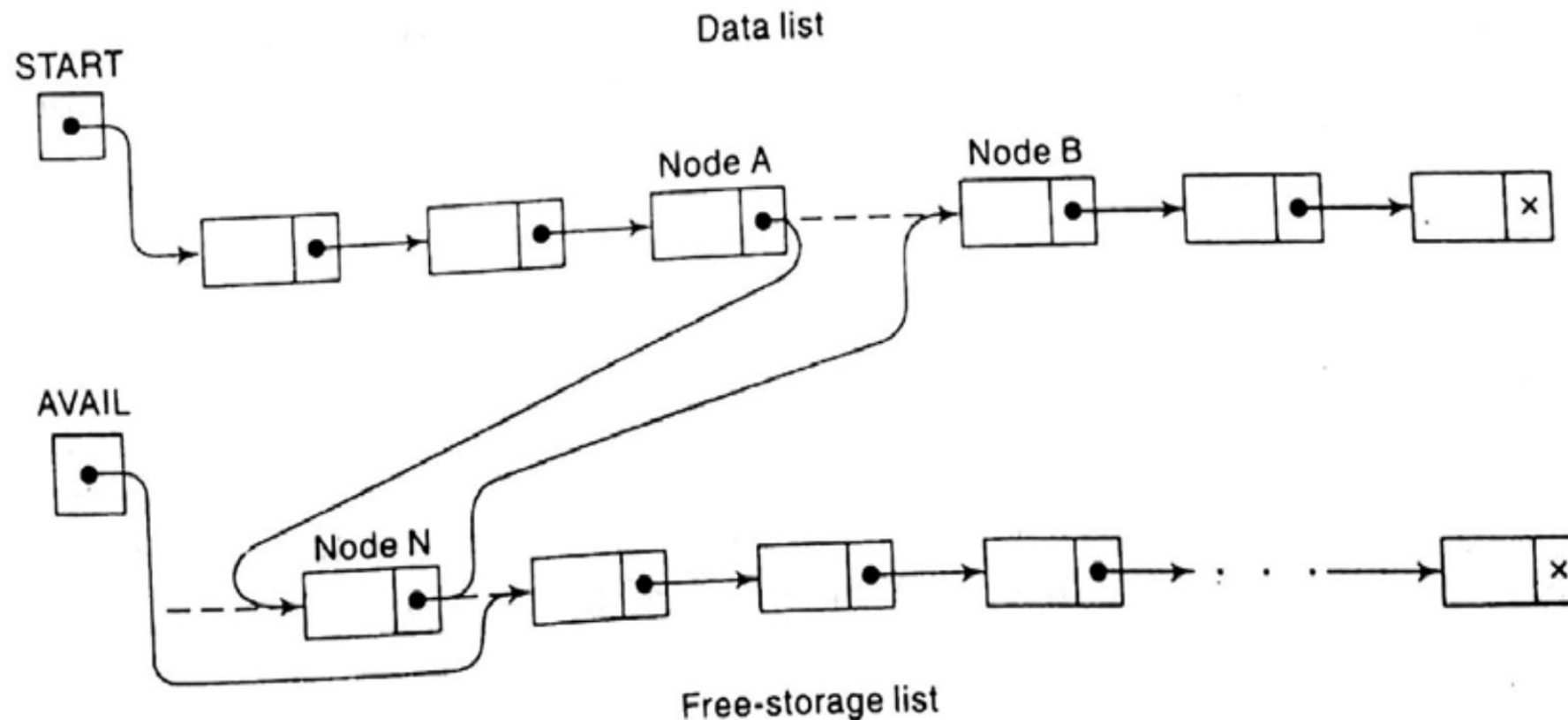


(a) Before insertion

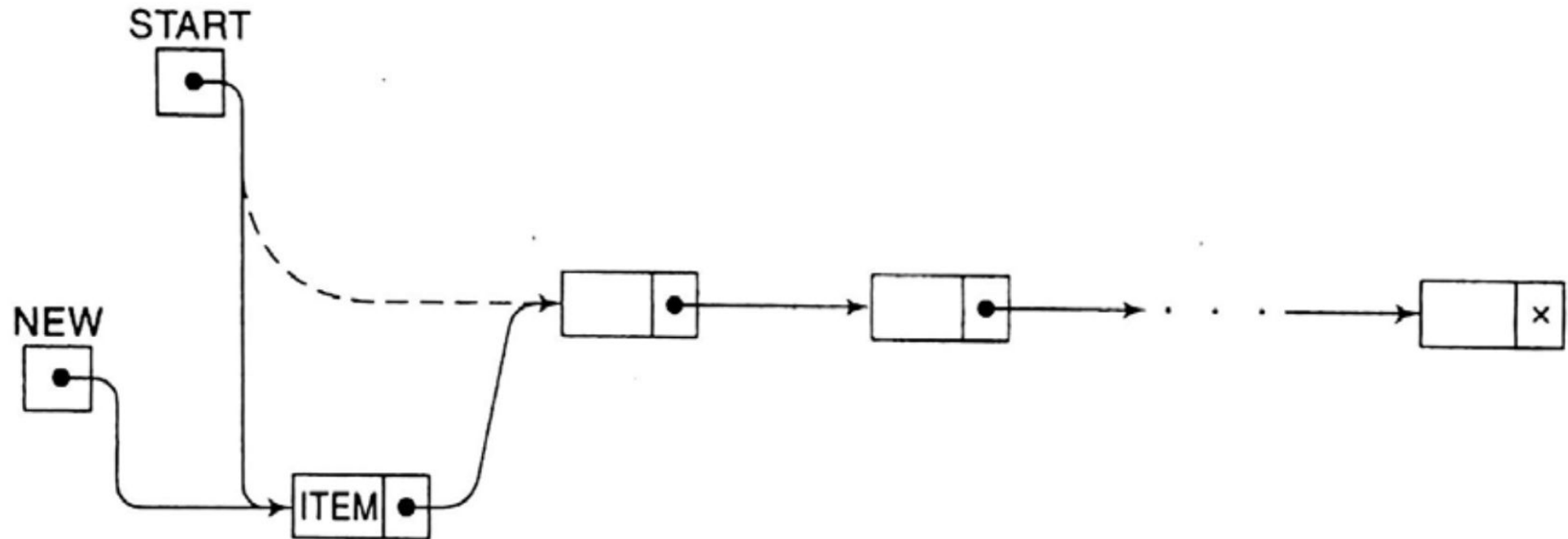


(b) After insertion

Insertion in Linked List [1]



Insertion at Beginning of the Linked List [1]





Linked List

◆ Linked List using C++:

```
struct Node
{
    int info;
    struct Node *ptr;
};
```

```
int main()
{
    struct Node* start = NULL;
    insertatBeginning(&start, 100);
    insertatBeginning(&start, 200);
    cout<<"The elements of linked list are : "<<endl;
    displayLinkedlist(start);
    return 0;
}
```

```
void insertatBeginning(struct Node** start, int newnode_info)
{
    struct Node* newnode = new Node;

    newnode->info = newnode_info;

    newnode->ptr = (*start);

    (*start) = newnode;
}
```

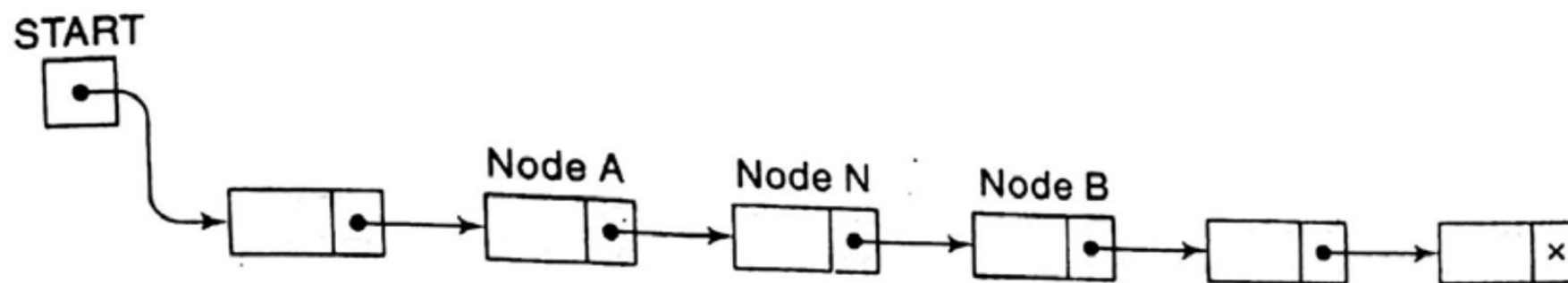
Insertion after a given node in Linked List [1]



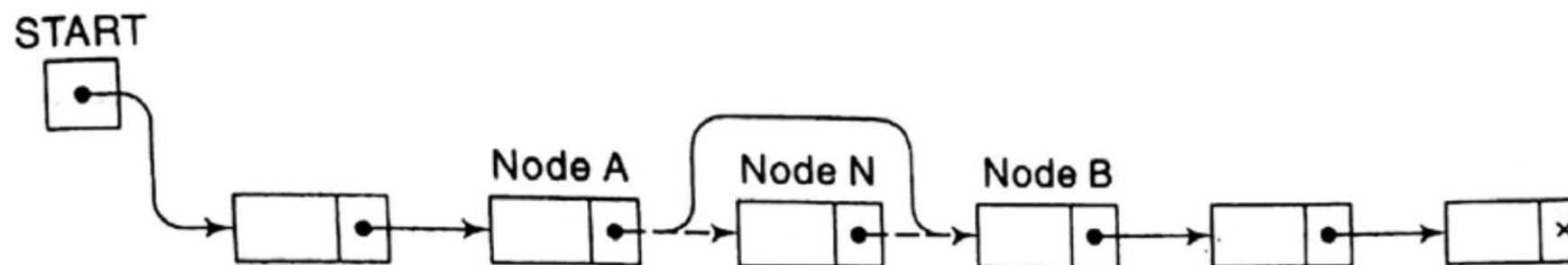
INSLOC(INFO, LINK, START, AVAIL, LOC, ITEM)

1. [OVERFLOW?] If AVAIL = NULL, then: Write: OVERFLOW, and Exit.
2. [Remove first node from AVAIL list.]
Set NEW := AVAIL and AVAIL := LINK[AVAIL].
3. Set INFO[NEW] := ITEM. [Copies new data into new node.]
4. If LOC = NULL, then: [Insert as first node.]
Set LINK[NEW] := START and START := NEW.
Else: [Insert after node with location LOC.]
Set LINK [NEW] := LINK[LOC] and LINK[LOC] := NEW.
[End of If structure.]
5. Exit.

Deletion from A Linked List [1]

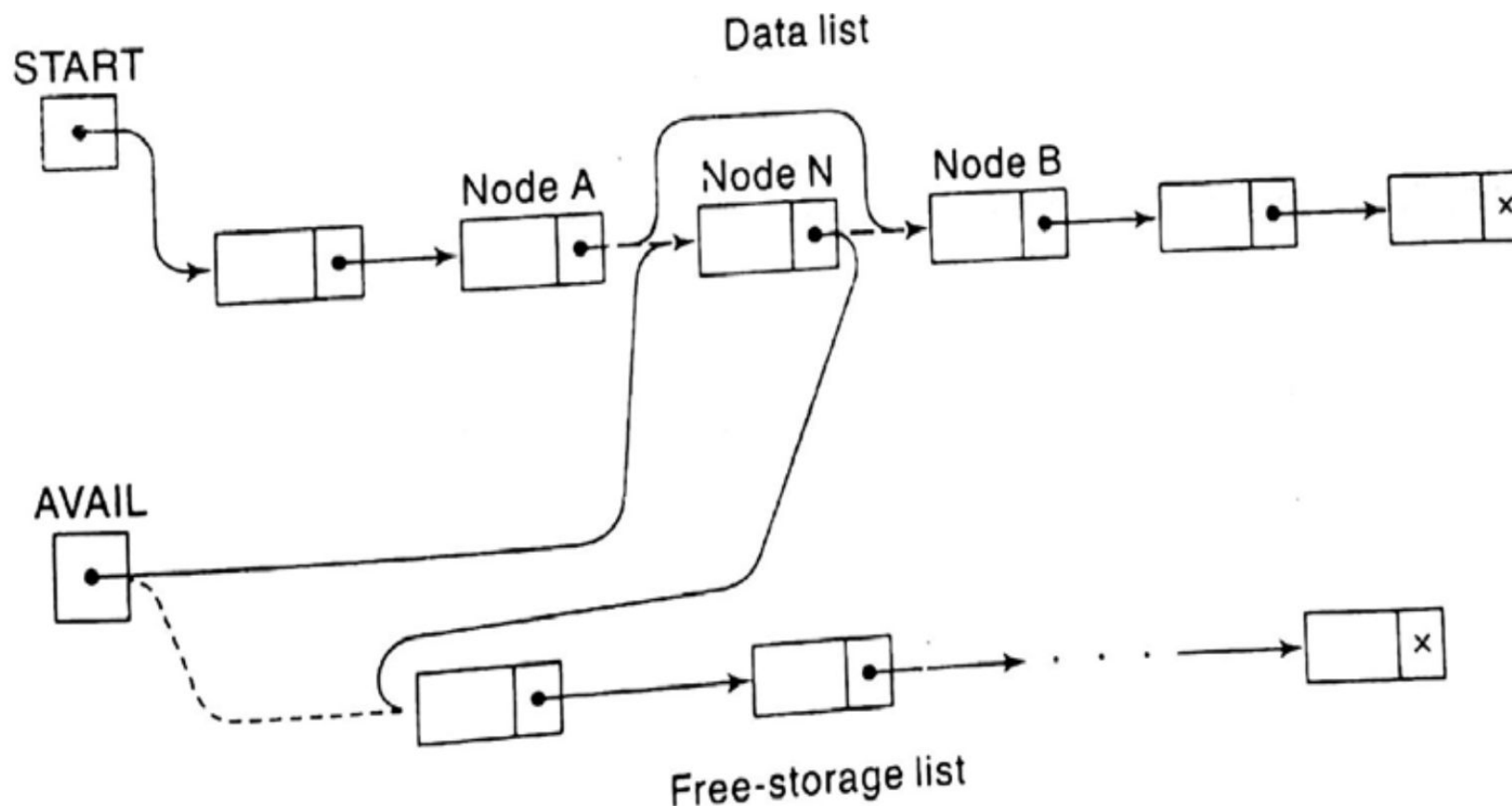


(a) Before deletion



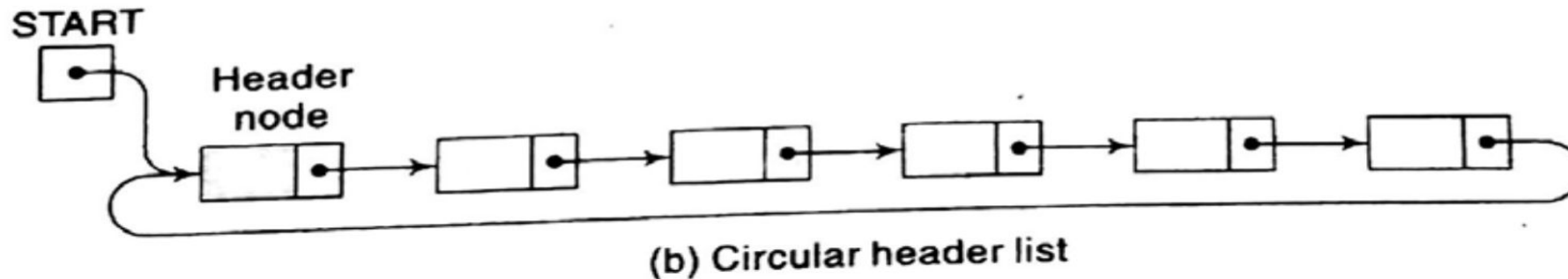
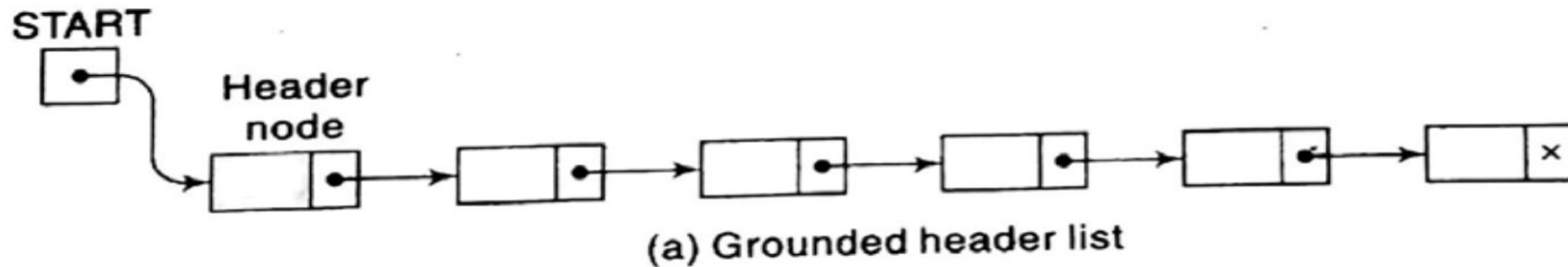
(b) After deletion

Deletion from A Linked List [1]

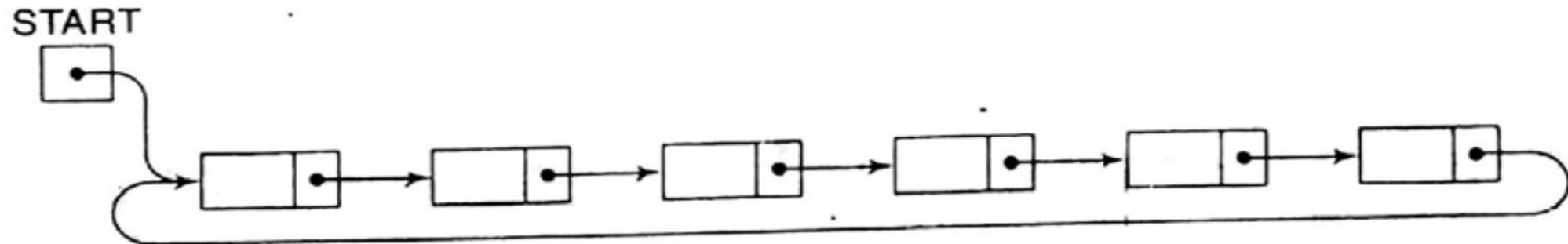


Header Linked List [1]

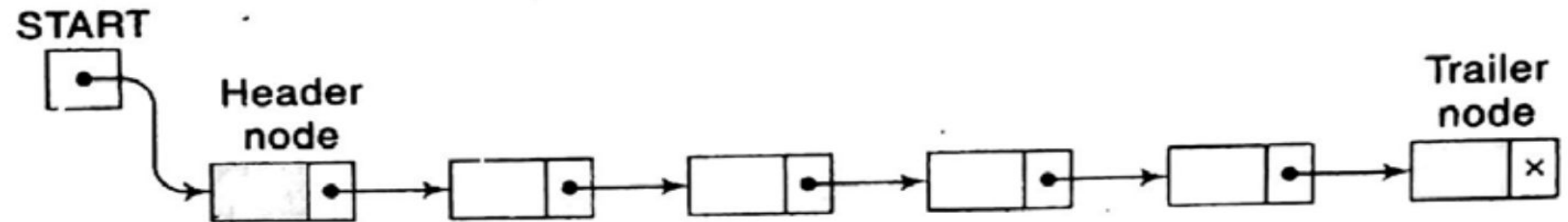
- ❖ A header linked list is a linked list which always contains **special node**, called header node, at the beginning of the list.



Linked List [1]

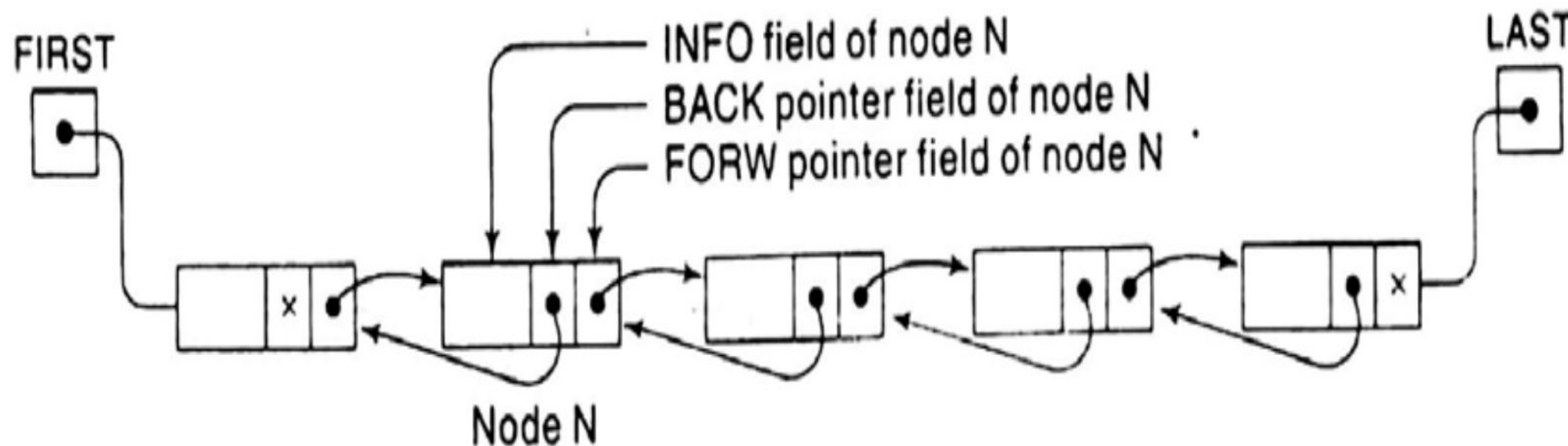


(a) Circular linked list

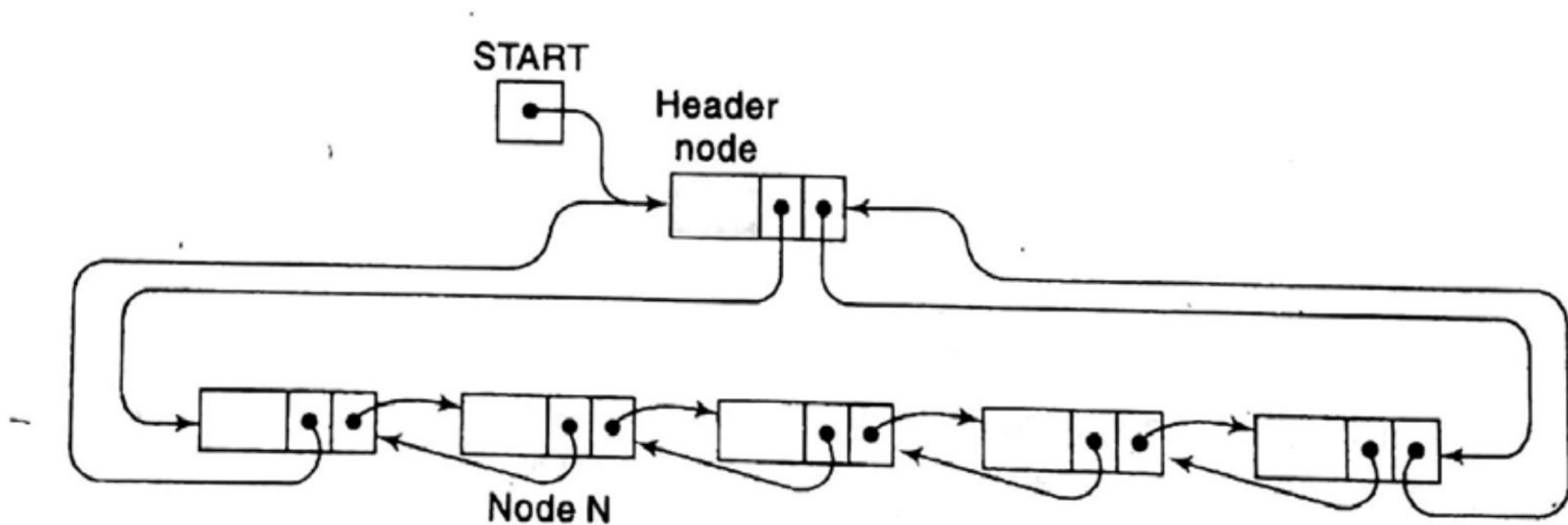


(b) Linked list with header and trailer nodes

Two way Lists [1]



Two way Header Lists [1]



Problems based on Linked List



- **Problem1:** Count number of times a given int occurs in the list

Problems based on Linked List



- **Problem:** How do you find the middle element of a singly linked list in one pass?

Problems based on Linked List



- **Problem:** How do you check if a given linked list contains a cycle?
How do you find the starting node of the cycle

Problems based on Linked List



- **Problem2:** A program takes a List and an integer index as input and return data value in the node at that index position.
- Example: {77, 12, 96} and input index is 2 then it should return 12.

Problems based on Linked List



- **Problem 3:** Given a list, split it into two sublist as Front sub-list and Back sub-list, if odd number of element then add last element into front list.

Problems based on Linked List



- **Problem 4:** Remove Duplicate Node from list

Problems based on Linked List



- **Problem 5:** Alternating Split the given list.

Problems based on Linked List



- **Problem 6:** Reverse the given list in single pass.

Problems based on Linked List



- **Problem 7:** Represent the following polynomial equation using Linked List

$$10x^3 + 6x^2 + 3x + 10$$

and add these two polynomial equation using Linked List:

$$6x^2 + 3x + 10$$

$$5x^2 + 7x + 2$$

Problems based on Linked List



- **Problem:** Given a Sorted doubly linked list of positive integers and an integer, then finds all the pairs (sum of two nodes data part) that is equal to the given integer value.
- **Example:** Double Linked List 2, 5, 7, 8, 9, 10, 12, 16, 19, 25, and $P=35$
- **Pairs will be (10, 25), (16, 19)**



References

-
1. Seymour Lipschutz, “Data Structures”, Schaum’s Series McGraw Hill edition 2013.
 2. <https://en.wikipedia.org/>